

LAPORAN PRATIKUM STRUKTUR DATA

PEKAN 8

disusun Oleh:

Habib Al Faruq

2511532010

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: Muhammad Galid Avero



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum struktur data mengenai penggunaan Algoritma Sorting. Penulisan laporan ini bertujuan untuk membuat sebuah laporan tentang materi pemrograman sort di Java yang meliputi Merge Sort, Quick Sort, dan Shell Sort

Penyusunan laporan praktikum ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada dosen pengampu mata kuliah praktikum pemrograman algoritma dan pemrograman, Bapak Dr. Wahyudi. S.T.M.T dan asisten praktikum, Uda Muhammad Galid Averro yang telah memberikan arahan dan masukan dalam proses penyusunan laporan praktikum ini. Penulis berharap laporan praktikum ini dapat memberikan manfaat bagi pembaca dalam mempelajari konsep penggunaan Algoritma Sorting di Java.

DAFTAR ISI

KATA PENGANTAR	1
DAFTAR ISI	2
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang Praktikum	3
1.2 Tujuan Praktikum	3
BAB II PEMBAHASAN.....	5
2.1 MergeSort_2511532010	5
2.2 QuickSort_2511532010	9
2.3 ShellSort_2511532010.....	12
2.4 BubbleSortGUI_2511532010.....	15
BAB III KESIMPULAN	20
3.1 Ringkasan Hasil Praktikum.....	20
DAFTAR PUSTAKA.....	21
TUGAS	22

BAB I PENDAHULUAN

1.1 Latar Belakang Praktikum

Pengurutan data (*sorting*) merupakan salah satu operasi paling mendasar dan krusial dalam ilmu komputer untuk meningkatkan efisiensi pencarian serta pengolahan data. Seiring dengan berkembangnya kompleksitas data, pemahaman terhadap algoritma dasar seperti *Bubble Sort* perlu ditingkatkan ke metode yang lebih efisien dan kompleks, seperti *Merge Sort* dan *Quick Sort* yang menggunakan pendekatan *divide and conquer*, serta *Shell Sort* yang mengoptimalkan perbandingan jarak jauh. Selain penerapan logikanya dalam bentuk berbasis teks, visualisasi proses eksekusi algoritma secara langkah demi langkah melalui Antarmuka Grafis Pengguna (*Graphical User Interface / GUI*) menggunakan Java Swing menjadi penting untuk mempermudah analisis pergerakan data, pelacakan indeks, dan pemahaman mekanisme kerja internal dari masing-masing algoritma tersebut secara riil.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah untuk mengimplementasikan dan memahami mekanisme kerja dari berbagai algoritma pengurutan tingkat lanjut, meliputi *Merge Sort*, *Quick Sort* dengan metode *median-of-three*, dan *Shell Sort* ke dalam bahasa pemrograman Java. Selain itu, praktikum ini bertujuan agar praktikan mampu membangun aplikasi simulator interaktif berbasis GUI (Java Swing) untuk memvisualisasikan proses perbandingan dan pertukaran elemen pada algoritma *Bubble Sort* secara runtut. Melalui implementasi ini, praktikan diharapkan dapat menganalisis perbedaan struktur perulangan, kondisi batas (*base case*) rekursif, serta logika manipulasi indeks memori dari masing-masing metode pengurutan.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini adalah meningkatkan kemampuan praktis praktikan dalam menulis kode program Java yang bersih dan terstruktur untuk menyelesaikan masalah komputasi kompleks. Praktikan dapat memahami secara mendalam kapan harus menerapkan struktur rekursif seperti pada *Merge Sort* dan *Quick Sort*, atau struktur iteratif dengan jarak variabel pada *Shell Sort*. Di samping itu, keterampilan dalam merancang tata letak (*Layout Manager*), mengelola komponen grafis, dan menangani *event listener* pada Java Swing memberikan pengalaman langsung yang berharga dalam membangun aplikasi desktop interaktif yang informatif serta mudah dipahami oleh pengguna.

BAB II PEMBAHASAN

2.1 MergeSort_2511532010

```
1 package pekan8_2511532010;
2
3 public class MergeSort_2511532010 {
4
5     void merge_2010(int[] arr_2010, int l_2010, int m_2010, int r_2010) {
6
7         int n1_2010 = m_2010 - l_2010 + 1;
8         int n2_2010 = r_2010 - m_2010;
9
10        int[] L_2010 = new int[n1_2010];
11        int[] R_2010 = new int[n2_2010];
12
13        for (int i_2010 = 0; i_2010 < n1_2010; ++i_2010) {
14            L_2010[i_2010] = arr_2010[l_2010 + i_2010];
15        }
16        for (int j_2010 = 0; j_2010 < n2_2010; ++j_2010) {
17            R_2010[j_2010] = arr_2010[m_2010 + 1 + j_2010];
18        }
19
20        int i_2010 = 0, j_2010 = 0;
21        int k_2010 = l_2010;
22
23        while (i_2010 < n1_2010 && j_2010 < n2_2010) {
24            if (L_2010[i_2010] <= R_2010[j_2010]) {
25                arr_2010[k_2010] = L_2010[i_2010];
26                i_2010++;
27            } else {
28                arr_2010[k_2010] = R_2010[j_2010];
29                j_2010++;
30            }
31            k_2010++;
32        }
33
34        while (i_2010 < n1_2010) {
35            arr_2010[k_2010] = L_2010[i_2010];
36            i_2010++;
37            k_2010++;
38        }
39
40        while (j_2010 < n2_2010) {
41            arr_2010[k_2010] = R_2010[j_2010];
42            j_2010++;
43            k_2010++;
44        }
45    }
46
47    void sort(int arr_2010[], int l_2010, int r_2010) {
48        if (l_2010 < r_2010) {
49            int m_2010 = (l_2010 + r_2010) / 2;
50
51            sort(arr_2010, l_2010, m_2010);
52
53            // PERBAIKAN: Mengganti m_2010 + l_2010 menjadi m_2010 + 1
54            sort(arr_2010, m_2010 + 1, r_2010);
55
56            merge_2010(arr_2010, l_2010, m_2010, r_2010);
57        }
58    }
59
60    static void printArray(int[] arr_2010) {
61        int n_2010 = arr_2010.length;
62        for (int i_2010 = 0; i_2010 < n_2010; ++i_2010) {
63            System.out.print(arr_2010[i_2010] + " ");
64        }
65        System.out.println();
66    }
67
68    public static void main(String[] args_2010) {
69        int[] arr_2010 = {12, 11, 13, 5, 6, 7};
70
71        System.out.println("Sebelum terurut:");
72        printArray(arr_2010);
73
74        MergeSort_2511532010 ob_2010 = new MergeSort_2511532010();
75        ob_2010.sort(arr_2010, 0, arr_2010.length - 1);
76
77        System.out.println("\nSesudah Terurut menggunakan Merge Sort:");
78        printArray(arr_2010);
79    }
80 }
```

1. Deklarasi Package dan Kelas

Baris awal menetapkan kode ini ke dalam package `pekan8_2511532010` dan mendefinisikan kelas utama `MergeSort_2511532010` yang membungkus seluruh logika algoritma pengurutan.

2. Penghitungan Ukuran Sub-array (`merge_2010`)

Variabel `n1_2010` menghitung jumlah elemen untuk sub-array bagian kiri, sementara `n2_2010` menghitung jumlah elemen untuk sub-array bagian kanan berdasarkan posisi indeks awal (`l_2010`), tengah (`m_2010`), dan akhir (`r_2010`).

3. Alokasi Memori Array Temporer

Baris `int[] L_2010` dan `int[] R_2010` membuat dua array baru di dalam memori sebagai tempat penyimpanan sementara untuk menyalin data sebelum proses perbandingan dilakukan.

4. Penyaliran Data ke Sub-array Kiri

Perulangan `for` pertama menyalin elemen-elemen dari array utama (`arr_2010`) dimulai dari indeks kiri (`l_2010`) sebanyak `n1_2010` elemen ke dalam array temporer `L_2010`.

5. Penyalinan Data ke Sub-array Kanan

Perulangan `for` kedua menyalin elemen-elemen dari array utama setelah posisi tengah (`m_2010 + 1`) sebanyak `n2_2010` elemen ke dalam array temporer `R_2010`.

6. Inisialisasi Indeks Penggabungan

Variabel `i_2010` dan `j_2010` diatur ke nilai 0 untuk melacak posisi elemen di array temporer `L_2010` dan `R_2010`, sedangkan `k_2010` diset ke posisi awal `l_2010` untuk melacak posisi penulisan pada array asli.

7. Logika Perbandingan dan Pengurutan

Perulangan `while` pertama membandingkan nilai elemen pada `L_2010[i_2010]` dengan `R_2010[j_2010]`. Elemen yang nilainya lebih kecil atau sama dengan akan langsung dimasukkan kembali ke `arr_2010[k_2010]` secara berurutan (*Ascending*).

8. Pembersihan Sisa Elemen Kiri

Perulangan while kedua bertugas memindahkan seluruh sisa elemen yang masih tertinggal di dalam array temporer L_2010 langsung ke array utama apabila isi elemen array R_2010 sudah habis dibandingkan terlebih dahulu.

9. Pembersihan Sisa Elemen Kanan

Perulangan while ketiga melakukan tugas yang sama untuk memindahkan sisa elemen dari array temporer R_2010 langsung ke array utama jika elemen di array L_2010 sudah habis.

10. Kondisi Batas Pembelahan (sort)

Pernyataan `if (l_2010 < r_2010)` berfungsi sebagai *base case* (kondisi berhenti) untuk proses rekursif. Pembelahan data hanya akan terjadi selama sub-array masih memiliki lebih dari satu elemen.

11. Penentuan Titik Tengah

Operasi hitung `int m_2010 = (l_2010 + r_2010) / 2` membagi dua wilayah indeks array saat ini secara seimbang untuk menentukan batas kiri dan kanan.

12. Rekursi Pembelahan Bagian Kiri

Baris `sort(arr_2010, l_2010, m_2010)` memanggil kembali fungsi `sort` secara rekursif untuk terus membelah sub-array bagian kiri hingga mencapai ukuran terkecil tunggal.

13. Rekursi Pembelahan Bagian Kanan

Baris `sort(arr_2010, m_2010 + 1, r_2010)` memanggil fungsi `sort` untuk membelah sub-array bagian kanan, dimulai dari satu indeks setelah titik tengah hingga indeks akhir.

14. Pemicu Penggabungan (Merge)

Setelah sub-array kiri dan kanan selesai dipecah dan disusun di tingkat rekursi terdalam, fungsi `merge_2010(...)` dipanggil untuk menggabungkan kembali kedua pecahan tersebut menjadi satu kesatuan array yang sudah terurut.

15. Fungsi Cetak (printArray)

Method statis ini menggunakan perulangan untuk menyusuri isi array dari indeks awal hingga akhir, lalu mencetaknya ke layar konsol secara horizontal yang dipisahkan oleh spasi.

16. Metode Utama (main)

Berperan sebagai pintu masuk program untuk menginisialisasi array acak {12, 11, 13, 5, 6, 7}, mencetak kondisi sebelum diurutkan, membuat objek kelas, mengeksekusi metode sort dari indeks 0 hingga panjang array dikurang 1, lalu menampilkan hasil akhir pengurutan.

2.2 QuickSort_2511532010

```
1 package pekan8_2511532010;
2 import java.util.*;
3 public class QuickSort_2511532010 {
4     static void swap (int[] arr, int i_2010, int j_2010)
5     {
6         int temp_2010 = arr[i_2010];
7         arr[i_2010] = arr[j_2010];
8         arr[j_2010]=temp_2010;
9     }
10
11     static void medianOfThree (int[] arr, int low_2010, int high_2010)
12     {
13         int mid_2010 = low_2010 + (high_2010-low_2010)/2;
14         if (arr[low_2010] > arr[mid_2010]) {
15             swap(arr, low_2010, mid_2010);
16         }
17         if (arr[low_2010] > arr[high_2010]) {
18             swap(arr, low_2010, high_2010);
19         }
20         if (arr[mid_2010] > arr[high_2010]) {
21             swap(arr, mid_2010, high_2010);
22         }
23         swap (arr, mid_2010, high_2010);
24     }
25
26     static int partition (int[] arr, int low_2010, int high_2010)
27     {
28         medianOfThree(arr, low_2010, high_2010);
29         int pivot_2010 = arr[high_2010];
30         int i_2010 = (low_2010 - 1);
31
32         for (int j_2010 = low_2010; j_2010<= high_2010-1; j_2010++) {
33             if (arr[j_2010] < pivot_2010) {
34                 i_2010++;
35                 swap(arr, i_2010, j_2010);
36             }
37         }
38         swap (arr, i_2010+1, high_2010);
39         return (i_2010 + 1);
40     }
41
42     static void QuickSort (int[]arr, int low_2010, int high_2010) {
43         if (low_2010< high_2010) {
44             int pi_2010 = partition (arr, low_2010, high_2010);
45             QuickSort(arr, low_2010, pi_2010-1);
46             QuickSort(arr, pi_2010 + 1, high_2010);
47         }
48     }
49
50     public static void printArr(int[] arr)
51     {
52         for (int i_2010 = 0; i_2010<arr.length; i_2010++) {
53             System.out.print(arr[i_2010]+ " ");
54         }
55         System.out.println();
56     }
57
58     public static void main(String[]args)
59     {
60         int[] arr = {10,7,8,9,1,5};
61         int N_2010 = arr.length;
62         System.out.print("Data sebelum diurutkan : ");
63         printArr(arr);
64
65         QuickSort (arr, 0, N_2010 - 1);
66
67         System.out.print("Data Terurut Quicksort : ");
68         printArr(arr);
69     }
70 }
71
```

1. Deklarasi Package dan Kelas

Baris awal menetapkan kode ini ke dalam package `pekan8_2511532010` dan mendefinisikan kelas utama `QuickSort_2511532010` sebagai wadah seluruh logika algoritma.

2. Method Pertukaran (swap)

Fungsi ini menerima sebuah array beserta dua indeks (`i_2010` dan `j_2010`), lalu menukar nilai posisi kedua elemen tersebut di dalam memori menggunakan bantuan variabel temporer `temp_2010`.

3. Penentuan Indeks Tengah (medianOfThree)

Fungsi ini mencari titik tengah array dengan rumus $\text{low_2010} + (\text{high_2010} - \text{low_2010}) / 2$ untuk menghindari risiko *integer overflow*.

4. Pengurutan Tiga Titik Elemen

Fungsi `medianOfThree` melakukan tiga tahap pengkondisian `if` untuk membandingkan nilai pada indeks awal (`low_2010`), tengah (`mid_2010`), dan akhir (`high_2010`), lalu mengurutkannya sehingga elemen dengan nilai terkecil berada di kiri dan nilai terbesar berada di kanan.

5. Penempatan Elemen Median ke Posisi Akhir

Baris terakhir di dalam `medianOfThree` menukar nilai tengah (`median`) yang berada di indeks `mid_2010` ke posisi indeks `high_2010` agar siap dijadikan sebagai nilai acuan (*pivot*) pada proses partisi.

6. Inisialisasi Proses Partisi (partition)

Fungsi ini memicu pencarian pivot optimal lewat `medianOfThree` terlebih dahulu, kemudian menyimpan nilai di indeks paling kanan (`high_2010`) ke dalam variabel `pivot_2010`.

7. Penanda Batas Elemen Kecil

Variabel `i_2010` diatur pada posisi awal `low_2010 - 1` untuk melacak batas posisi terakhir dari elemen array yang nilainya terbukti lebih kecil daripada nilai pivot.

8. Perulangan Pemindaian Array

Struktur `for` bergerak menyusuri elemen array dari indeks `low_2010` hingga indeks sebelum pivot (`high_2010 - 1`).

9. Logika Pemindahan Elemen

Jika elemen saat ini (`arr[j_2010]`) bernilai lebih kecil dari `pivot_2010`, maka indeks `i_2010` akan bergeser naik satu tingkat, dan elemen tersebut langsung ditukar ke area sebelah kiri melalui fungsi `swap`.

10. Penempatan Akhir Posisi Pivot

Setelah perulangan selesai, elemen pivot yang berada di ujung kanan (`high_2010`) ditukar dengan elemen di posisi `i_2010 + 1` agar nilai pivot berada tepat di tengah-tengah, memisahkan kelompok angka kecil di kiri dan kelompok angka besar di kanan.

11. Pengembalian Titik Sekat

Fungsi `partition` mengembalikan nilai indeks `i_2010 + 1` sebagai posisi final pivot yang akan digunakan sebagai acuan pembelahan rekursif berikutnya.

12. Kondisi Batas Pembelahan (QuickSort)

Pernyataan `if (low_2010 < high_2010)` bertindak sebagai batas berhenti (*base case*) proses rekursif agar pembagian berhenti ketika sub-array sudah berukuran satu elemen atau kosong.

13. Rekursi Sisi Kiri

Baris `QuickSort(arr, low_2010, pi_2010 - 1)` memanggil kembali fungsi secara rekursif untuk menyelesaikan pengurutan kelompok angka yang berada di sebelah kiri pivot.

14. Rekursi Sisi Kanan

Baris `QuickSort(arr, pi_2010 + 1, high_2010)` memanggil kembali fungsi secara rekursif untuk menyelesaikan pengurutan kelompok angka yang berada di sebelah kanan pivot.

15. Method Cetak Array (`printArr`)

Fungsi pembantu untuk melakukan iterasi menyusuri isi elemen array dari awal hingga akhir untuk dicetak secara horizontal ke layar konsol yang dipisahkan spasi.

16. Metode Utama (`main`)

Berperan sebagai pintu masuk program untuk mendeklarasikan data array acak `{10, 7, 8, 9, 1, 5}`, mengambil panjang total array, menampilkan isi data sebelum

diproses, mengeksekusi metode QuickSort dari indeks 0 hingga indeks terakhir, lalu menampilkan hasil akhir data yang telah terurut.

2.3 ShellSort_2511532010

```
1 package pekan8_2511532010;
2 import java.util.*;
3
4 public class ShellSort_2511532010 {
5     public static void shellsort(int[] A_2010) {
6         int n_2010 = A_2010.length;
7         int gap_2010 = n_2010 / 2;
8
9         while (gap_2010 > 0) {
10             for (int i_2010 = gap_2010; i_2010 < n_2010; i_2010++) {
11                 int temp_2010 = A_2010[i_2010];
12
13                 int j_2010 = i_2010;
14
15                 while (j_2010 >= gap_2010 && A_2010[j_2010 - gap_2010] > temp_2010) {
16                     A_2010[j_2010] = A_2010[j_2010 - gap_2010];
17                     j_2010 = j_2010 - gap_2010;
18                 }
19                 A_2010[j_2010] = temp_2010;
20             }
21             gap_2010 = gap_2010 / 2;
22         }
23     }
24 }
25
26 public static void main(String[] args_2010) {
27     int[] data_2010 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
28
29     System.out.print("Sebelum: ");
30     printArray(data_2010);
31
32     shellsort(data_2010);
33
34     System.out.print("Sesudah (Shell Sort): ");
35     printArray(data_2010);
36 }
37
38 public static void printArray(int[] arr_2010) {
39     for (int i_2010 : arr_2010) {
40         System.out.print(i_2010 + " ");
41     }
42     System.out.println();
43 }
44 }
```

1. Deklarasi Package dan Kelas

Baris awal menetapkan kode ini ke dalam package pekan8_2511532010 dan mendefinisikan kelas utama ShellSort_2511532010 yang membungkus semua logika pengurutan.

2. Inisialisasi Panjang Array

Variabel `int n_2010 = A_2010.length` digunakan untuk mengambil jumlah total elemen yang ada di dalam array `A_2010`.

3. Penentuan Jarak Awal (Gap)

Variabel `int gap_2010 = n_2010 / 2` mendefinisikan jarak awal antar elemen yang akan dibandingkan, yang dihitung dengan membagi dua panjang total array.

4. Perulangan Kontrol Jarak (Gap Loop)

Struktur `while (gap_2010 > 0)` bertindak sebagai loop terluar yang memastikan proses pengurutan terus berjalan selama ukuran jarak antar elemen lebih besar dari nol.

5. Perulangan Pemindaian Elemen

Struktur `for (int i_2010 = gap_2010; i_2010 < n_2010; i_2010++)` memulai penyisipan elemen dari indeks sebesar nilai `gap_2010` berjalan maju sampai akhir array.

6. Penyimpanan Elemen Sementara

Baris `int temp_2010 = A_2010[i_2010]` mengamankan nilai elemen aktif saat ini ke dalam variabel temporer agar nilainya tidak hilang saat terjadi pergeseran data.

7. Inisialisasi Indeks Pembanding

Variabel `int j_2010 = i_2010` menyalin posisi indeks aktif untuk digunakan sebagai pelacak posisi dalam proses perbandingan dan pergeseran mundur.

8. Evaluasi Jarak Mundur

Perulangan `while` bagian dalam memeriksa kondisi apakah indeks `j_2010` masih dalam jangkauan jarak dan apakah elemen di posisi kiri (`A_2010[j_2010 - gap_2010]`) nilainya lebih besar dari nilai acuan `temp_2010`.

9. Pergeseran Elemen ke Kanan

Baris `A_2010[j_2010] = A_2010[j_2010 - gap_2010]` menduplikasi nilai elemen yang lebih besar ke posisi kanan sejauh ukuran `gap` yang ditentukan.

10. Pengurangan Indeks Pelacak

Perintah `j_2010 = j_2010 - gap_2010` menggeser posisi indeks pelacak mundur ke kiri sejauh ukuran `gap` untuk melakukan pemeriksaan pada elemen-elemen sebelumnya dalam sub-list yang sama.

11. Penempatan Elemen Acuan

Setelah kondisi loop pergeseran mundur tidak lagi terpenuhi, nilai yang tersimpan di `temp_2010` dimasukkan ke posisi akhirnya di `A_2010[j_2010]`.

12. Reduksi Ukuran Jarak

Perintah `gap_2010 = gap_2010 / 2` memperkecil ukuran jarak menjadi setengahnya di setiap akhir putaran besar untuk merapatkan elemen yang dibandingkan sampai gap bernilai 1 (sama dengan Insertion Sort standar).

13. Fungsi Cetak Array (`printArray`)

Method pembantu menggunakan perulangan *for-each* untuk menyusuri setiap elemen di dalam array dan menampilkannya secara horizontal ke layar konsol dengan pemisah spasi.

14. Metode Utama (`main`)

Berperan sebagai titik masuk program untuk mendeklarasikan array acak `data_2010` berisi 10 elemen, menampilkan data sebelum diurutkan, mengeksekusi method `shellsort`, dan menampilkan hasil akhir array yang telah terurut.

2.4 BubbleSortGUI_2511532010

```
1 package pekan8_2511532010;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.Font;
7
8 import javax.swing.BorderFactory;
9 import javax.swing.JButton;
10 import javax.swing.JFrame;
11 import javax.swing.JLabel;
12 import javax.swing.JOptionPane;
13 import javax.swing.JPanel;
14 import javax.swing.JScrollPane;
15 import javax.swing.JTextArea;
16 import javax.swing.JTextField;
17 import javax.swing.SwingConstants;
18 import javax.swing.SwingUtilities;
19 import javax.swing.border.EmptyBorder;
20
21 public class BubbleSortGUI_2511532010 extends JFrame {
22
23     private static final long serialVersionUID = 1L;
24     private int[] array_2010;
25     private JLabel[] labelArray_2010;
26     private JButton stepButton_2010, resetButton_2010, setButton_2010;
27     private JTextField inputField_2010;
28     private JPanel panelArray_2010;
29     private JTextArea stepArea_2010;
30
31     private int i_2010 = 1, j_2010;
32     private boolean sorting_2010 = false;
33     private int stepCount_2010 = 1;
34
35     /**
36      * Create the frame.
37      */
38     public BubbleSortGUI_2511532010() {
39         setTitle("Insertion Sort Langkah per Langkah");
40         setSize(1000, 450);
41         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
42         setLocationRelativeTo(null);
43         setLayout(new BorderLayout());
44
45         JPanel inputPanel = new JPanel(new BorderLayout(10, 0));
46         inputPanel.setBorder(new EmptyBorder(10, 10, 10, 10));
47
48         inputField_2010 = new JTextField();
49         setButton_2010 = new JButton("Set Array");
50
51         inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma) : ", BorderLayout.WEST);
52         inputPanel.add(inputField_2010, BorderLayout.CENTER);
53         inputPanel.add(setButton_2010, BorderLayout.EAST);
54
55         // Panel array visual
56         panelArray_2010 = new JPanel();
57         panelArray_2010.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.CENTER, 10, 50));
58
59         // Panel kontrol
60         JPanel controlPanel = new JPanel();
61         stepButton_2010 = new JButton("Langkah Selanjutnya");
62         resetButton_2010 = new JButton("Reset");
63
64         stepButton_2010.setEnabled(false);
65         controlPanel.add(stepButton_2010);
66         controlPanel.add(resetButton_2010);
67
68         // Area teks untuk log langkah-langkah
69         stepArea_2010 = new JTextArea(8, 50); // Sedikit dikurangi agar seimbang
70         stepArea_2010.setEditable(false);
71         stepArea_2010.setFont(new Font("Monospaced", Font.PLAIN, 14));
72
73         JScrollPane scrollPane_2010 = new JScrollPane(stepArea_2010);
74         scrollPane_2010.setBorder(BorderFactory.createTitledBorder("Log Proses Sorting"));
75
76         add(inputPanel, BorderLayout.NORTH);
77         add(panelArray_2010, BorderLayout.CENTER);
78         add(controlPanel, BorderLayout.SOUTH);
79         add(scrollPane_2010, BorderLayout.EAST);
80
81         // Event Set Array
82         setButton_2010.addActionListener(g -> setArrayFromInput_2010());
83
84         // Event Langkah Selanjutnya
85         stepButton_2010.addActionListener(g -> performStep_2010());
86
87         // Event Reset
88         resetButton_2010.addActionListener(g -> reset_2010());
89     }
90
91     private void setArrayFromInput_2010() {
92         String text_2010 = inputField_2010.getText().trim();
93
94         if (text_2010.isEmpty())
95             return;
96     }
```



```

97     String[] parts_2010 = text_2010.split(",");
98
99     array_2010 = new int[parts_2010.length];
100
101     try {
102         for (int k_2010 = 0; k_2010 < parts_2010.length; k_2010++) {
103             array_2010[k_2010] = Integer.parseInt(parts_2010[k_2010].trim());
104         }
105     } catch (NumberFormatException e_2010) {
106         JOptionPane.showMessageDialog(
107             this,
108             "Masukkan hanya angka yang dipisahkan dengan koma!",
109             "Error",
110             JOptionPane.ERROR_MESSAGE
111         );
112         return;
113     }
114
115     i_2010 = 0;
116     j_2010 = 0;
117     stepCount_2010 = 1;
118     sorting_2010 = true;
119
120     stepButton_2010.setEnabled(true);
121     stepArea_2010.setText("");
122     panelArray_2010.removeAll();
123
124     labelArray_2010 = new JLabel[array_2010.length];
125
126     for (int k_2010 = 0; k_2010 < array_2010.length; k_2010++) {
127         labelArray_2010[k_2010] = new JLabel(String.valueOf(array_2010[k_2010]));
128         labelArray_2010[k_2010].setFont(new Font("Arial", Font.BOLD, 24));
129         labelArray_2010[k_2010].setOpaque(true);
130         labelArray_2010[k_2010].setBackground(Color.WHITE);
131         labelArray_2010[k_2010].setBorder(BorderFactory.createLineBorder(Color.BLACK));
132         labelArray_2010[k_2010].setPreferredSize(new Dimension(50, 50));
133         labelArray_2010[k_2010].setHorizontalAlignment(SwingConstants.CENTER);
134
135         panelArray_2010.add(labelArray_2010[k_2010]);
136     }
137
138     panelArray_2010.revalidate();
139     panelArray_2010.repaint();
140
141
142     private void performStep_2010() {
143
144         if (!sorting_2010 || i_2010 >= array_2010.length - 1) {
145             sorting_2010 = false;
146             stepButton_2010.setEnabled(false);
147             JOptionPane.showMessageDialog(this, "Sorting selesai!");
148             return;
149         }
150         resetHighlights_2010();
151
152         StringBuilder stepLog_2010 = new StringBuilder();
153
154         labelArray_2010[j_2010].setBackground(Color.CYAN);
155         labelArray_2010[j_2010 + 1].setBackground(Color.CYAN);
156
157         if (array_2010[j_2010] > array_2010[j_2010 + 1]) {
158             int temp_2010 = array_2010[j_2010];
159             array_2010[j_2010] = array_2010[j_2010 + 1];
160             array_2010[j_2010 + 1] = temp_2010;
161
162             labelArray_2010[j_2010].setBackground(Color.RED);
163             labelArray_2010[j_2010 + 1].setBackground(Color.RED);
164
165             stepLog_2010.append("Langkah ").append(stepCount_2010).append(": Menukar elemen ke-")
166                 .append(j_2010).append(" ")
167                 .append(array_2010[j_2010 + 1]).append(" dengan ke-")
168                 .append(j_2010 + 1).append(" ")
169                 .append(array_2010[j_2010]).append("\n");
170         } else {
171             stepLog_2010.append("Langkah ").append(stepCount_2010).append(": Tidak ada pertukaran antara ke-")
172                 .append(j_2010).append(" dan ke-")
173                 .append(j_2010 + 1).append("\n");
174         }
175
176         stepLog_2010.append("Hasil: ").append(arrayToString_2010(array_2010)).append("\n\n");
177         stepArea_2010.append(stepLog_2010.toString());
178
179         updateLabels_2010();
180
181         j_2010++;
182
183         if (j_2010 >= array_2010.length - i_2010 - 1) {
184             j_2010 = 0;
185             i_2010++;
186         }
187
188         stepCount_2010++;
189
190         if (i_2010 >= array_2010.length - 1) {
191             sorting_2010 = false;

```

```

192         stepButton_2010.setEnabled(false);
193         JOptionPane.showMessageDialog(this, "Sorting selesai!");
194     }
195 }
196 private void updateLabels_2010() {
197     for (int k_2010 = 0; k_2010 < array_2010.length; k_2010++) {
198         labelArray_2010[k_2010].setText(String.valueOf(array_2010[k_2010]));
199     }
200 }
201 private void resetHighlights_2010() {
202     for (JLabel label_2010 : labelArray_2010) {
203         label_2010.setBackground(Color.WHITE);
204     }
205 }
206 }
207 private void reset_2010() {
208     inputField_2010.setText("");
209     panelArray_2010.removeAll();
210     panelArray_2010.revalidate();
211     panelArray_2010.repaint();
212     stepArea_2010.setText("");
213     stepButton_2010.setEnabled(false);
214
215     sorting_2010 = false;
216     i_2010 = 0;
217     j_2010 = 0;
218     stepCount_2010 = 1;
219 }
220 private String arrayToString_2010(int[] arr_2010) {
221     StringBuilder sb_2010 = new StringBuilder();
222     for (int k_2010 = 0; k_2010 < arr_2010.length; k_2010++) {
223         sb_2010.append(arr_2010[k_2010]);
224
225         if (k_2010 < arr_2010.length - 1) {
226             sb_2010.append(", ");
227         }
228     }
229     return sb_2010.toString();
230 }
231 public static void main(String[] args_2010) {
232     SwingUtilities.invokeLater(() -> {
233         BubbleSortGUI_2511532010 gui_2010 = new BubbleSortGUI_2511532010();
234         gui_2010.setVisible(true);
235     });
236 }
237 }

```

1. Deklarasi Kelas dan Kerangka Jendela

Kelas BubbleSortGUI_2511532010 mewarisi kelas JFrame untuk bertindak sebagai komponen jendela utama aplikasi visualisasi pengurutan data angka.

2. Deklarasi Atribut State

Struktur internal kelas menampung array integer array_2010, array komponen labelArray_2010 (JLabel) untuk representasi kotak visual angka, tombol kontrol, area teks log, serta variabel penanda langkah perulangan (i_2010, j_2010, stepCount_2010).

3. Tata Letak Komponen Grafis (Constructor)

Komponen diatur menggunakan BorderLayout sebagai kerangka utama, di mana bagian atas (NORTH) diisi panel input teks, bagian tengah (CENTER) diisi panel visualisasi kotak angka, bagian bawah (SOUTH) diisi tombol kontrol langkah, dan bagian kanan (EAST) diisi area log berbasis JScrollPane.

4. Registrasi Aksi Listener

Tombol Set Array, Langkah Selanjutnya, dan Reset dihubungkan langsung ke method fungsional masing-masing menggunakan ekspresi lambda untuk menangani event klik dari pengguna.

5. Parsing Teks Input (setArrayFromInput_2010)

Method ini mengambil string dari kotak input, memotong spasi menggunakan `.trim()`, dan memecah teks berdasarkan tanda koma menggunakan `.split(",")` untuk dialokasikan ke dalam array integer.

6. Proteksi Validasi Format Angka

Proses konversi String ke Integer menggunakan `Integer.parseInt()` dibungkus dalam blok try-catch untuk menangkap `NumberFormatException` dan menampilkan kotak dialog error jika ada karakter non-angka yang dimasukkan.

7. Pembangkitan Kotak Visual Array

Setelah data berhasil diparsing, program mengosongkan panel lama, membuat instans objek JLabel baru sebanyak jumlah elemen array, mengatur ukuran kotak (50x50), memberi bingkai garis hitam, mengaktifkan sifat opasitas (`setOpaque(true)`), dan menyusunnya secara horizontal.

8. Validasi Batas Akhir Langkah (performStep_2010)

Di awal setiap penekanan tombol langkah, program memeriksa apakah pengurutan telah selesai (`i_2010 >= array_2010.length - 1`). Jika benar, tombol dinonaktifkan dan pop-up informasi penyelesaian ditampilkan.

9. Penyorotan Elemen Aktif

Ketika proses perbandingan dilakukan, warna latar belakang dua kotak JLabel pada indeks aktif `j_2010` dan `j_2010 + 1` diubah menjadi warna cyan untuk menandai bahwa kedua angka tersebut sedang dibandingkan.

10. Eksekusi Pertukaran Nilai dan Penandaan Merah:

ika nilai `array_2010[j_2010]` lebih besar dari `array_2010[j_2010 + 1]`, nilai tersebut ditukar menggunakan variabel bantuan `temp_2010`, latar belakang kedua kotak diubah menjadi merah (tanda terjadi swap), dan informasi penukaran dicatat ke log string.

11. Pencatatan Langkah Tanpa Pertukaran

Jika kondisi perbandingan bernilai salah (tidak perlu ditukar), program masuk ke blok else untuk mencatat dalam log teks bahwa tidak ada pertukaran elemen pada langkah tersebut.

12. Sinkronisasi Data Nilai ke Label (`updateLabels_2010`)

Setelah kalkulasi nilai array berubah akibat pertukaran, method ini dipanggil untuk memperbarui teks angka yang tertulis di dalam komponen visual JLabel agar sesuai dengan kondisi riil di memori.

13. Manajemen Pergeseran Indeks Bubble Sort

Indeks pembanding `j_2010` dinaikkan satu tingkat. Jika `j_2010` telah mencapai batas akhir putaran (`array_2010.length - i_2010 - 1`), posisi `j_2010` direset kembali ke 0 dan indeks perulangan luar `i_2010` dinaikkan satu tingkat.

14. Normalisasi Warna Kotak (`resetHighlights_2010`)

Fungsi ini menggunakan perulangan *for-each* untuk mengembalikan warna latar belakang seluruh objek JLabel kembali menjadi putih sebelum langkah perbandingan berikutnya dimulai.

15. Fitur Reset Keadaan (`reset_2010`)

Method ini membersihkan kolom teks input, menghapus seluruh komponen visual dari panel utama, mengosongkan teks pada area log langkah, menonaktifkan tombol eksekusi, serta mengembalikan seluruh variabel kontrol perulangan ke posisi awal.

16. Konversi Array ke String (`arrayToString_2010`)

Fungsi pembantu (*helper*) yang mengiterasi array integer dan merangkainya menjadi string yang dipisahkan koma untuk kebutuhan pencetakan baris hasil di area log.

17. Eksekusi Utama Jendela (`main`)

Merupakan pintu masuk program untuk memicu instansiasi kelas GUI di dalam perintah `SwingUtilities.invokeLater()` agar pembuatan antarmuka grafis berjalan aman di dalam thread khusus grafis (*Event Dispatch Thread*).

BAB III KESIMPULAN

3.1 Ringkasan Hasil Praktikum

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa masing-masing algoritma pengurutan memiliki karakteristik, cara pembagian sub-array, dan penanganan indeks memori yang unik. Algoritma *Merge Sort* bekerja secara konsisten dengan membelah array secara rekursif menjadi sub-array terkecil lalu menggabungkannya kembali secara terurut memanfaatkan array temporer. Di sisi lain, *Quick Sort* mengandalkan efisiensi pemilihan *pivot* yang dioptimalkan dengan metode *median-of-three* untuk membagi data menjadi area elemen kecil dan besar sebelum melakukan partisi rekursif. Sementara itu, *Shell Sort* berhasil menjembatani kelemahan *Insertion Sort* konvensional dengan memanfaatkan sistem *gap* (jarak variabel) yang terus mengecil secara bertahap untuk mempercepat pergeseran elemen acak berjarak jauh.

Selain itu, implementasi visualisasi pengurutan berbasis GUI menggunakan Java Swing berhasil membuktikan bahwa pelacakan status program (*state tracing*) menjadi jauh lebih intuitif dibandingkan output teks konvensional. Melalui manipulasi komponen JLabel, pengaturan warna latar belakang (seperti warna *cyan* untuk perbandingan dan warna merah untuk pertukaran), serta pencatatan otomatis pada log JTextArea, alur berpikir dari algoritma *Bubble Sort* dapat diamati secara transparan pada setiap perubahan indeks *i* dan *j*. Integrasi antara logika pemrograman murni dan desain antarmuka grafis ini sangat efektif dalam mempermudah proses analisis jalannya eksekusi program serta deteksi kesalahan logika (*debugging*) selama proses pengembangan perangkat lunak.

DAFTAR PUSTAKA

- [1] Oracle, *The Java® Tutorials — Primitive Data Types*. [Online]. Available: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>
- [2] GeeksforGeeks, *Java Data Types – GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/data-types-in-java>

TUGAS

1. Lagu_2511532010

```
1 package pekan8_2511532010;
2
3 //Simpan file ini dengan nama: Lagu_2010.java
4 public class Lagu_2511532010 {
5     String judul_2010;
6     String penyanyi_2010;
7     int durasi_2010;
8
9     public Lagu_2511532010(String judul_2010, String penyanyi_2010, int durasi_2010) {
10         this.judul_2010 = judul_2010;
11         this.penyanyi_2010 = penyanyi_2010;
12         this.durasi_2010 = durasi_2010;
13     }
14 }
```

2. Sorting_2511532010

```

1 package pekan8_2511532010;
2
3 //Simpan file ini dengan nama: Sorting_2511532010.java
4 import java.util.Scanner;
5
6 public class Sorting_2511532010 {
7
8     // Array penyimpanan maksimal 20 lagu dengan tipe data lagu 2511532010
9     static Lagu_2511532010[] dataLagu_2010 = new Lagu_2511532010[20];
10     static int jumlahLagu_2010 = 0;
11
12     // Method untuk mengisi data secara manual dari input user
13     public static void inputData_2010(Scanner scanner_2010) {
14         System.out.print("\nMasukkan jumlah lagu yang ingin diinput (maks 20): ");
15         jumlahLagu_2010 = scanner_2010.nextInt();
16         scanner_2010.nextLine(); // Mengembalikan buffer enter
17
18         if (jumlahLagu_2010 > 20) {
19             jumlahLagu_2010 = 20;
20             System.out.println("Batas maksimal adalah 20. Program akan membatasi input menjadi 20 lagu.");
21         }
22
23         for (int i_2010 = 0; i_2010 < jumlahLagu_2010; i_2010++) {
24             System.out.println("Lagu ke-" + (i_2010 + 1));
25             System.out.print("Judul : ");
26             String judul_2010 = scanner_2010.nextLine();
27
28             System.out.print("Penyanyi : ");
29             String penyanyi_2010 = scanner_2010.nextLine();
30
31             System.out.print("Durasi (detik) : ");
32             int durasi_2010 = scanner_2010.nextInt();
33             scanner_2010.nextLine(); // Mengembalikan buffer enter
34
35             // Inisialisasi objek baru sesuai dengan nama class yang baru
36             dataLagu_2010[i_2010] = new Lagu_2511532010(judul_2010, penyanyi_2010, durasi_2010);
37             System.out.println();
38         }
39     }
40
41     // Method untuk menampilkan sebelum dan sesudah sorting
42     public static void tampilData_2010(String pesan_2010) {
43         System.out.println(pesan_2010);
44         for (int i_2010 = 0; i_2010 < jumlahLagu_2010; i_2010++) {
45             System.out.println((i_2010 + 1) + ". " + dataLagu_2010[i_2010].judul_2010 + " - " + dataLagu_2010[i_2010].durasi_2010 + " detik");
46         }
47     }
48
49     // Algoritma 1: Shell Sort (Berdasarkan Judul A-Z)
50     public static void shellSort_2010() {
51         for (int gap_2010 = jumlahLagu_2010 / 2; gap_2010 > 0; gap_2010 /= 2) {
52             for (int i_2010 = gap_2010; i_2010 < jumlahLagu_2010; i_2010++) {
53                 Lagu_2511532010 temp_2010 = dataLagu_2010[i_2010];
54                 int j_2010;
55                 for (j_2010 = i_2010; j_2010 >= gap_2010 && dataLagu_2010[j_2010 - gap_2010].judul_2010.compareToIgnoreCase(temp_2010.judul_2010) > 0; j_2010 -= gap_2010) {
56                     dataLagu_2010[j_2010] = dataLagu_2010[j_2010 - gap_2010];
57                 }
58                 dataLagu_2010[j_2010] = temp_2010;
59             }
60         }
61     }
62
63     // Algoritma 2: Quick Sort (Berdasarkan Durasi Ascending)
64     public static void quickSort_2010(int low_2010, int high_2010) {
65         if (low_2010 < high_2010) {
66             int pi_2010 = partition_2010(low_2010, high_2010);
67             quickSort_2010(low_2010, pi_2010 - 1);
68             quickSort_2010(pi_2010 + 1, high_2010);
69         }
70     }
71
72     // Partition untuk Quick Sort
73     private static int partition_2010(int low_2010, int high_2010) {
74         int pivot_2010 = dataLagu_2010[high_2010].durasi_2010;
75         int i_2010 = (low_2010 - 1);
76         for (int j_2010 = low_2010; j_2010 < high_2010; j_2010++) {
77             if (dataLagu_2010[j_2010].durasi_2010 <= pivot_2010) {
78                 i_2010++;
79                 Lagu_2511532010 temp_2010 = dataLagu_2010[i_2010];
80                 dataLagu_2010[i_2010] = dataLagu_2010[j_2010];
81                 dataLagu_2010[j_2010] = temp_2010;
82             }
83         }
84         Lagu_2511532010 temp_2010 = dataLagu_2010[i_2010 + 1];
85         dataLagu_2010[i_2010 + 1] = dataLagu_2010[high_2010];
86         dataLagu_2010[high_2010] = temp_2010;
87         return i_2010 + 1;
88     }
89 }

```



```

90 // Algoritma 3: Merge Sort (Berdasarkan Judul A-Z)
91 public static void mergeSort_2010(int left_2010, int right_2010) {
92     if (left_2010 < right_2010) {
93         int mid_2010 = left_2010 + (right_2010 - left_2010) / 2;
94         mergeSort_2010(left_2010, mid_2010);
95         mergeSort_2010(mid_2010 + 1, right_2010);
96         merge_2010(left_2010, mid_2010, right_2010);
97     }
98 }
99
100 // Proses penggabungan untuk Merge Sort
101 private static void merge_2010(int left_2010, int mid_2010, int right_2010) {
102     int n1_2010 = mid_2010 - left_2010 + 1;
103     int n2_2010 = right_2010 - mid_2010;
104
105     Lagu_2511532010[] L_2010 = new Lagu_2511532010[n1_2010];
106     Lagu_2511532010[] R_2010 = new Lagu_2511532010[n2_2010];
107
108     for (int i_2010 = 0; i_2010 < n1_2010; ++i_2010) {
109         L_2010[i_2010] = dataLagu_2010[left_2010 + i_2010];
110     }
111     for (int j_2010 = 0; j_2010 < n2_2010; ++j_2010) {
112         R_2010[j_2010] = dataLagu_2010[mid_2010 + 1 + j_2010];
113     }
114
115     int i_2010 = 0, j_2010 = 0;
116     int k_2010 = left_2010;
117
118     while (i_2010 < n1_2010 && j_2010 < n2_2010) {
119         if (L_2010[i_2010].judul_2010.compareToIgnoreCase(R_2010[j_2010].judul_2010) <= 0) {
120             dataLagu_2010[k_2010] = L_2010[i_2010];
121             i_2010++;
122         } else {
123             dataLagu_2010[k_2010] = R_2010[j_2010];
124             j_2010++;
125         }
126         k_2010++;
127     }
128
129     while (i_2010 < n1_2010) {
130         dataLagu_2010[k_2010] = L_2010[i_2010];
131         i_2010++;
132         k_2010++;
133     }
134
135     while (j_2010 < n2_2010) {
136         dataLagu_2010[k_2010] = R_2010[j_2010];
137         j_2010++;
138         k_2010++;
139     }
140 }
141
142 public static void main(String[] args) {
143     Scanner scanner_2010 = new Scanner(System.in);
144
145     System.out.println("=== Sorting Playlist NIM: 2511532010 ===");
146     System.out.print("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
147     int pilihan_2010 = scanner_2010.nextInt();
148
149     // Panggil method input dengan mengirimkan scanner
150     inputData_2010(scanner_2010);
151
152     tampilData_2010("Data Sebelum Sorting:");
153
154     switch (pilihan_2010) {
155         case 1:
156             shellSort_2010();
157             tampilData_2010("\nData Setelah Shell Sort (Judul A-Z):");
158             break;
159         case 2:
160             quickSort_2010(0, jumlahLagu_2010 - 1);
161             tampilData_2010("\nData Setelah Quick Sort (Durasi Asc):");
162             break;
163         case 3:
164             mergeSort_2010(0, jumlahLagu_2010 - 1);
165             tampilData_2010("\nData Setelah Merge Sort (Judul A-Z):");
166             break;
167         default:
168             System.out.println("\nPilihan tidak valid. Silakan jalankan ulang program.");
169     }
170
171     scanner_2010.close();
172 }
173 }

```

HASIL :

```
<terminated> Sorting_2511532010 [Java Application] /Library/Java/JavaV
=== Sorting Playlist NIM: 2511532010 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Masukkan jumlah lagu yang ingin diinput (maks 20): 3
Lagu ke-1
Judul      : Abracadabra
Penyanyi   : Lady Gaga
Durasi (detik) : 223

Lagu ke-2
Judul      : Copycat
Penyanyi   : Billie Eilish
Durasi (detik) : 193

Lagu ke-3
Judul      : Driver License
Penyanyi   : Olivia Rodrigo
Durasi (detik) : 242
|
Data Sebelum Sorting:
1. Abracadabra - 223 detik
2. Copycat - 193 detik
3. Driver License - 242 detik

Data Setelah Quick Sort (Durasi Asc):
1. Copycat - 193 detik
2. Abracadabra - 223 detik
3. Driver License - 242 detik
```

PENJELASAN :

1. Struktur File dan Package

Kode dibagi menjadi dua kelas di dalam package pekan8_2511532010. Kelas pertama Lagu_2511532010 bertindak sebagai struktur data (Model), sedangkan kelas Sorting_2511532010 berisi logika pengurutan dan metode utama.

2. Model Data Lagu

Kelas `Lagu_2511532010` memiliki tiga atribut: `judul_2010`, `penyanyi_2010`, dan `durasi_2010`. Sebuah *constructor* disediakan untuk mempermudah pembuatan objek baru dengan mengirimkan ketiga nilai tersebut secara langsung.

3. Inisialisasi Array Global

Di dalam kelas utama `Sorting_2511532010`, dideklarasikan sebuah array statis `dataLagu_2010` dengan kapasitas maksimal 20 lagu, serta variabel `jumlahLagu_2010` untuk melacak seberapa banyak array tersebut telah terisi.

4. Proses Input Data (`inputData_2010`)

Method ini meminta pengguna untuk memasukkan jumlah lagu yang diinginkan. Jika angka melebihi 20, program secara otomatis membatasinya menjadi 20.

5. Perulangan Pengisian Data

Di dalam method `inputData_2010`, sebuah perulangan `for` berjalan untuk meminta input judul, nama penyanyi, dan durasi dari pengguna secara berurutan. Setiap kali selesai mengisi satu lagu, program langsung membuat objek `Lagu_2511532010` baru dan menyimpannya ke dalam array.

6. Fungsi Menampilkan Data (`tampilData_2010`)

Method ini bertugas mencetak daftar lagu ke layar. Di dalamnya terdapat perulangan yang menyusuri array dan mencetak output dengan format yang diminta, yaitu: nomor. judul - durasi detik.

7. Logika Shell Sort (`shellSort_2010`)

Mengurutkan lagu berdasarkan Judul (A-Z). Metode ini menggunakan variabel `gap_2010` (jarak perbandingan) yang terus mengecil setengahnya hingga bernilai 1.

8. Perbandingan Teks pada Shell Sort

Dalam melakukan pengurutan abjad, algoritma ini memanfaatkan fungsi `compareToIgnoreCase()` untuk membandingkan antara judul lagu saat ini dengan judul lagu di posisi sebelumnya.

9. Logika Quick Sort (`quickSort_2010`)

Mengurutkan lagu berdasarkan Durasi (Ascending). Ini adalah fungsi rekursif yang terus membelah array menjadi sub-array kiri dan kanan berdasarkan nilai batas (*pivot*).

10. Proses Pemisahan Data (partition_2010)

Fungsi bantuan untuk Quick Sort. Fungsi ini mengambil durasi lagu terakhir sebagai elemen acuan (*pivot*). Lagu dengan durasi lebih kecil atau sama akan dikumpulkan di sisi kiri, sedangkan yang lebih besar di sisi kanan.

11. Logika Merge Sort (mergeSort_2010)

Mengurutkan lagu berdasarkan Judul (A-Z). Algoritma ini secara rekursif membagi total array menjadi dua bagian yang sama besar (kiri dan kanan) hingga tersisa elemen tunggal.

12. Proses Penggabungan (merge_2010)

Setelah array terbelah, fungsi ini menyalin sub-array ke tempat sementara (L_2010 dan R_2010). Kemudian, program membandingkan judul dari kedua wadah sementara tersebut (dengan `compareToIgnoreCase`) dan menyusunnya kembali ke array utama secara berurutan.

13. Metode Utama (main)

Titik awal berjalannya program. Langkah pertama adalah menampilkan judul tugas lalu meminta pengguna mengetikkan angka pilihan algoritma (1, 2, atau 3).

14. Eksekusi Alur Program

Setelah pengguna memilih algoritma, method `inputData_2010` dipanggil agar pengguna dapat memasukkan detail lagu. Setelah selesai, program mencetak data mentah lewat `tampilData_2010`.

15. Pemrosesan Percabangan (switch)

Berdasarkan input awal pengguna, struktur *switch-case* akan menentukan algoritma mana yang akan dieksekusi (Shell, Quick, atau Merge).

16. Penyelesaian

Setelah data berhasil diurutkan oleh algoritma yang terpilih, program kembali memanggil `tampilData_2010` untuk mencetak hasil akhir playlist lagu yang telah disusun dengan rapi.